

Linux Server Operations Lab

Phase 5 Review

AnsibleによるLinux日次点検の自動化

1. Phase 5 の目的

Phase 5では、Ansibleを使って3台のLinuxサーバに対する日次点検を自動化しました。対象は uptime、メモリ、ディスク、OS、IPアドレス、nginx、CPU です。

ここまでのPhase 1～4で「Excelの台帳をPythonで読み、Ansible inventoryへ変換して3台へ接続する」流れを作成済みです。Phase 5では、その先の運用作業としてサーバ点検をまとめて実行できるようにしました。

2. 対象サーバ

ServerID	Hostname	IP	用途
SV001	tokyo	192.168.56.11	web
SV002	yokohama	192.168.56.12	web
SV003	omiya	192.168.56.13	web

3. 最初に行った手動確認

Playbook化する前に、Ansibleのad hocコマンドで各項目を個別に確認しました。いきなり大きなPlaybookを書くのではなく、コマンド単位で期待した結果が返るかを確認してから自動化しました。

```
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'uptime'
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'free -m'
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'df -h /'
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'cat /etc/os-release'
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'ip -br addr'
ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini -a 'systemctl is-active nginx'
```

確認できた代表値

項目	確認結果
メモリ	各サーバ約961MB
ディスク /	tokyo 25%、yokohama 16%、omiya 16%
OS	3台とも Ubuntu 24.04.4 LTS
nginx	3台とも active
CPU	topで確認。いずれも低負荷

4. daily_check.yml の作成

確認済みのコマンドをPlaybookにまとめ、1回のansible-playbook実行で3台をまとめて点検できるようにしました。点検処理なので、変更が発生したように見せないため `changed_when: false` を設定しています。

```
---
- name: Daily server check
  hosts: servers
  gather_facts: false

  tasks:
    - name: Check uptime
      ansible.builtin.command: uptime
      register: uptime_result
      changed_when: false

    - name: Show uptime
      ansible.builtin.debug:
        var: uptime_result.stdout

    - name: Check memory
      ansible.builtin.command: free -m
      register: memory_result
      changed_when: false

    - name: Show memory
      ansible.builtin.debug:
        var: memory_result.stdout

    - name: Check disk
      ansible.builtin.command: df -h /
      register: disk_result
      changed_when: false

    - name: Show disk
      ansible.builtin.debug:
        var: disk_result.stdout

    - name: Check OS
      ansible.builtin.command: cat /etc/os-release
      register: os_result
      changed_when: false

    - name: Show OS
      ansible.builtin.debug:
        var: os_result.stdout
```

Playbook後半では IP アドレス、nginx、CPU も同じ考え方で取得しています。CPU 確認はパイプを使うため `ansible.builtin.shell` を使用しました。

5. 実行コマンドと結果

```
ansible-playbook -i ~/linux-server-operations-lab/inventory/hosts_generated.ini \
~/linux-server-operations-lab/playbooks/daily_check.yml
```

PLAY RECAP

```
-----
omiya    : ok=14 changed=0 unreachable=0 failed=0
tokyo    : ok=14 changed=0 unreachable=0 failed=0
yokohama : ok=14 changed=0 unreachable=0 failed=0
```

3台すべてで `failed=0 / unreachable=0` となり、日次点検Playbookが正常に完了しました。`changed=0` なので、点検によってサーバ設定を変更していないことも確認できます。

6. このPhaseで理解したこと

Ansibleは設定変更だけでなく、複数サーバから状態を一括収集する用途にも使えます。特に `register` でコマンド結果を変数に保存し、`debug`で確認する流れは、後続のPython解析やExcel書き戻しにつながる重要な考え方でした。

command と shell の使い分け

単純なコマンド実行は `command` を優先し、パイプやリダイレクトなどシェル機能が必要な場合だけ `shell` を使う、という使い分けを確認しました。

changed_when: false の意味

点検はサーバを変更する処理ではありません。そのため `changed_when: false` を付けることで、実行結果を「変更なし」として扱い、運用上の見やすさを高めています。

7. トラブルシュート・注意点

CPU確認では `top -bn1 | grep` を使うため `shell` モジュールが必要です。また、サーバごとにネットワークインターフェースの表示数が違う場合があり、IP出力全体を単純比較するのではなく管理NICを意識して確認する必要があります。

8. 実務での応用

このPlaybookをcron、AWXのSchedule、監視・レポート処理と組み合わせれば、毎日のサーバ状態確認を自動化できます。人が3台へ個別SSHして同じコマンドを繰り返す作業を減らせます。

9. 次のPhaseへのつながり

Phase 5では「情報を取得する」ところまででした。Phase 6では、そのAnsible出力をPythonで解析し、OK / WARNING / FAILED / UNREACHABLEに分類する処理へ進みます。

10. note記事候補

「AnsibleでLinuxサーバ3台の日次点検を自動化する」「`register` / `changed_when` を実運用イメージで理解する」「手作業の点検をPlaybookへ置き換える」といった記事へ展開できます。